

SUPABASE

+

KOTLIN

Guía Completa: Integración de Supabase con Android Kotlin y ViewModel

Manual técnico · Diagramas de arquitectura · Flujo CRUD

Android Kotlin

Supabase

ViewModel

Jetpack Compose

MVVM

PostgreSQL

PostgREST

CRUD

Basado en la documentacion oficial de Supabase para Android Kotlin

supabase.com/docs/guides/getting-started/quickstarts/kotlin

Tabla de Contenidos

INTRODUCCION	4
PARTE I — MANUAL PASO A PASO	5
1. Idea central de la integracion	5
2. Crear el proyecto en Supabase	5
3. Crear la tabla y cargar datos de ejemplo	5
4. Politica RLS para lectura publica	6
5. Exponer la tabla en la Data API	7
6. Crear el proyecto Android Kotlin	7
7. Instalar dependencias	7
8. Permiso de Internet	8
9. Obtener URL y Publishable Key	8
10. Inicializar el cliente Supabase	9
11. Crear el modelo de datos	9
12. Llevar la consulta al ViewModel	9
13. Conectar la UI al ViewModel	10
14.Codigo minimo integrado	11
15. Ejecutar la aplicacion	13
16. Flujo resumido	13
17. Problemas mas probables	13
PARTE II — DIAGRAMA CRUD CON VIEWMODEL	15
1. Diagrama de arquitectura CRUD	15
2. Lectura de cada capa	16
3. Flujo de conexion paso a paso	17

4. Las cuatro operaciones CRUD	18
5. Seguridad y acceso	18
PARTE III — DIAGRAMA TECNICO DESDE ACTIVITY	20
1. Diagrama tecnico completo	20
2. Analisis de capas y componentes	20
3. Flujo tecnico de extremo a extremo	22
4. Servicios del backend Supabase	22
5. Patron MVVM en detalle	24
CONCLUSION GENERAL	25

Introduccion

Este documento reúne en un solo lugar todo lo necesario para integrar Supabase a una aplicación Android en Kotlin con arquitectura ViewModel: el manual técnico paso a paso, el diagrama de flujo CRUD y el diagrama técnico completo desde la Activity hasta el backend.

Supabase es una plataforma de backend de código abierto que provee base de datos PostgreSQL, autenticación, almacenamiento y funcionalidades en tiempo real a través de una API REST autogenerada. Android Kotlin, combinado con Jetpack Compose y el patrón ViewModel, permite construir aplicaciones móviles modernas que consumen esa API de forma limpia y mantenible.

La documentación oficial de Supabase para Android Kotlin presenta un quickstart funcional que realiza la consulta directamente desde el código de UI. La misma guía aclara que en producción se recomienda usar un ViewModel para separar la interfaz de la lógica de datos. Este documento toma esa recomendación y la desarrolla en tres dimensiones:

- **Práctica:** un manual con todos los pasos de configuración, código y solución de problemas.
- **Conceptual:** un diagrama de arquitectura que explica el flujo CRUD y el papel de cada capa.
- **Técnica:** un diagrama de flujo completo desde la Activity hasta los servicios del backend de Supabase.

FUENTE TECNICA BASE

La referencia técnica exclusiva para la integración es la documentación oficial de Supabase:

supabase.com/docs/guides/getting-started/quickstarts/kotlin

PARTE I

Manual Paso a Paso

Configuracion completa de Supabase y Android Kotlin con ViewModel

1. Idea central de la integracion

La conexion entre Supabase y una aplicacion Android con ViewModel puede entenderse como un flujo de cuatro capas claramente diferenciadas:

- 1 **Supabase** crea y expone los datos a traves de una API REST automatica sobre PostgreSQL.
- 2 **Android** obtiene permiso del sistema operativo para comunicarse con Internet.
- 3 La **app inicializa un cliente Supabase** que conoce la URL y la clave del proyecto.
- 4 El **ViewModel consulta los datos** a traves de ese cliente y la UI unicamente los representa como estado observado.

MODELO MENTAL

Supabase provee los datos → Android solicita conexion → el cliente traduce la intencion → el ViewModel coordina → la UI refleja el estado.

2. Crear el proyecto en Supabase

Ingresa a database.new y cree un proyecto nuevo en la plataforma de Supabase. El panel le pedira: nombre del proyecto, contrasena de la base de datos (guardela) y la region geografica del servidor.

IMPORTANTE

La creacion del proyecto puede tardar hasta dos minutos mientras se aprovisiona la base de datos PostgreSQL. Espere a que el estado cambie a **Active** antes de continuar.

3. Crear la tabla y cargar datos de ejemplo

Una vez que el proyecto este activo, abra el **SQL Editor** del panel de Supabase y ejecute el siguiente script oficial:

```
-- Crear la tabla
create table instruments (
  id   bigint primary key generated always as identity,
  name text not null
);

-- Insertar datos de ejemplo
insert into instruments (name)
values
  ('violin'),
  ('viola'),
  ('cello');

-- Otorgar permiso de lectura al rol anon
grant select on public.instruments to anon;

-- Activar Row Level Security
alter table instruments enable row level security;
```

Tabla 1 Descripción de las sentencias SQL del script de inicialización

Sentencia	Efecto
<code>create table instruments</code>	Crea la tabla con un id autoincremental y un campo name de texto.
<code>insert into instruments</code>	Inserta tres filas de ejemplo que se verán en la app.
<code>grant select ... to anon</code>	Otorga permisos de lectura al rol anonimo.
<code>alter table ... enable row level security</code>	Activa RLS para que el acceso este controlado por politicas.

4. Crear la politica RLS para lectura publica

Activar RLS sin politica deja la tabla protegida pero inaccesible. Ejecute la politica oficial en el SQL Editor:

```
-- Politica que permite al rol anon leer la tabla instruments
create policy "public can read instruments"
  on public.instruments
  for select to anon
  using (true);
```

PASO INDISPENSABLE

Sin esta politica, cualquier consulta desde la app devolvera una lista vacia aunque la tabla contenga datos y los permisos de `grant` esten correctamente configurados.

5. Exponer la tabla en la Data API

En el Dashboard de Supabase, vaya a **Integrations** > **Data API** y exponga la tabla `instruments` para que sea accesible a traves de PostgREST. Si desea que futuras tablas del esquema `public` queden expuestas automaticamente, puede habilitar *Default privileges for new entities*.

NOTA

Solo las tablas explicitamente expuestas en la Data API estaran disponibles. Las tablas no expuestas retornaran un error 404 desde el cliente Android.

6. Crear el proyecto Android Kotlin

En Android Studio, cree un proyecto nuevo:

1. Seleccione **New > New Android Project**.
2. Elija la plantilla **Empty Activity**.
3. Asigne un nombre y seleccione **Kotlin** como lenguaje.
4. Establezca la API minima recomendada (API 24 o superior).
5. Haga clic en **Finish** y espere a que Gradle sincronice.

7. Instalar dependencias

Abra el archivo `build.gradle.kts` del modulo `app` y agregue lo indicado por la guia oficial:

```

plugins {
    // ... otros plugins existentes
    kotlin("plugin.serialization") version "$kotlin_version"
}

dependencies {
    // ... otras dependencias existentes
    implementation(platform("io.github.jan-tennert.supabase:bom:$supabase_version"))
    implementation("io.github.jan-tennert.supabase:postgrest-kt")
    implementation("io.ktor:ktor-client-android:$ktor_version")
}

```

Tabla 2 Funcion de cada dependencia de Supabase para Android

Dependencia	Proposito
<code>kotlin("plugin.serialization")</code>	Serializa y deserializa datos entre JSON y objetos Kotlin mediante <code>@Serializable</code> .
<code>postgrest-kt</code>	Permite consultar tablas expuestas por Supabase a traves de PostgreSQL con una API tipada en Kotlin.
<code>ktor-client-android</code>	Provee el motor HTTP subyacente para realizar peticiones de red en Android.

8. Agregar permiso de Internet

En `AndroidManifest.xml`, dentro de `<manifest>` y fuera de `<application>`, agregue:

```
<uses-permission android:name="android.permission.INTERNET" />
```

SIN ESTE PERMISO LA APP NO PODRA CONECTARSE

Android deniega todo trafico de red por defecto. Sin declarar este permiso, todas las llamadas a Supabase fallaran silenciosamente o lanzaran una excepcion de red.

9. Obtener la URL y la Publishable Key del proyecto

Segun la documentacion oficial, debe obtener dos valores del panel de Supabase:

- **Project URL:** con formato `https://xyzcompany.supabase.co`.

- **Publishable key:** la clave publica para uso desde el cliente.

Estas se obtienen desde el dialogo **Connect** del proyecto en el Dashboard de Supabase.

PUNTO DE SEGURIDAD CRITICO

Para una aplicacion movil se debe usar exclusivamente la **Publishable key**. Las claves de servidor (`service_role`) tienen permisos de administrador y no deben nunca incluirse en codigo cliente.

10. Inicializar el cliente de Supabase

La guia oficial muestra la siguiente inicializacion del cliente, que debe realizarse una unica vez:

```
val supabase = createSupabaseClient(
    supabaseUrl = "https://xyzcompany.supabase.co",
    supabaseKey = "your_publishable_key"
) {
    install(Postgrest)
}
```

Los cuatro elementos indispensables de esta inicializacion son: `createSupabaseClient` , `supabaseUrl` , `supabaseKey` e `install(Postgrest)` .

11. Crear el modelo de datos

La documentacion define una clase serializable que representa cada fila de la tabla `instruments` :

```
@Serializable
data class Instrument(
    val id: Int,
    val name: String,
)
```

COINCIDENCIA EXACTA

Los nombres de los campos (`id` , `name`) deben coincidir exactamente con los nombres de las columnas en la tabla de Supabase. Una discrepancia causara un error de decodificacion en tiempo de ejecucion.

12. Llevar la consulta al ViewModel

La guía oficial explícitamente indica que en producción conviene usar un ViewModel. La adaptación mínima es la siguiente:

```
class InstrumentsViewModel : ViewModel() {

    var instruments by mutableStateOf<List<Instrument>>(emptyList())
    private set

    fun loadInstruments() {
        viewModelScope.launch {
            withContext(Dispatchers.IO) {
                instruments = supabase
                    .from("instruments")
                    .select()
                    .decodeList<Instrument>()
            }
        }
    }
}
```

Responsabilidades del ViewModel

- Conserva el estado de la lista `instruments` de forma sobreviviente a cambios de configuración.
- Ejecuta la consulta a Supabase en un hilo de IO sin bloquear el hilo principal.
- Transforma la respuesta JSON en una lista tipada de objetos `Instrument`.
- Expone el estado únicamente mediante el setter privado, preservando la encapsulación.

Tabla 3 Diferencia entre el quickstart oficial y la versión con ViewModel

Aspecto	Quickstart oficial	Con ViewModel
Quien consulta datos	La UI (composable)	El ViewModel
Quien mantiene estado	La UI	El ViewModel
Separación de responsabilidades	Mezclada (UI + datos)	Clara (UI solo representa)
Supervivencia a rotación	No garantizada	Si, por diseño

13. Conectar la UI al ViewModel

La interfaz deja de consultar la base de datos directamente y unicamente representa el estado que el ViewModel expone:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            SupabaseTutorialTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) { InstrumentsScreen() }
            }
        }
    }
}

@Composable
fun InstrumentsScreen(
    viewModel: InstrumentsViewModel = viewModel()
) {
    val instruments = viewModel.instruments
    LaunchedEffect(Unit) { viewModel.loadInstruments() }
    LazyColumn {
        items(instruments, key = { instrument -> instrument.id }) { instrument ->
            Text(text = instrument.name, modifier = Modifier.padding(8.dp))
        }
    }
}
```

14. Código mínimo integrado

El siguiente bloque reúne todos los componentes necesarios para una primera versión funcional:

```

// Inicializacion del cliente
val supabase = createSupabaseClient(
    supabaseUrl = "https://xyzcompany.supabase.co",
    supabaseKey = "your_publishable_key"
) { install(Postgrest) }

// Modelo de datos
@Serializable
data class Instrument(val id: Int, val name: String)

// ViewModel
class InstrumentsViewModel : ViewModel() {
    var instruments by mutableStateOf<List<Instrument>>(emptyList())
    private set
    fun loadInstruments() {
        viewModelScope.launch {
            withContext(Dispatchers.IO) {
                instruments =
supabase.from("instruments").select().decodeList<Instrument>()
            }
        }
    }
}

// Activity
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            SupabaseTutorialTheme {
                Surface(modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background) {
                    InstrumentsScreen()
                }
            }
        }
    }
}

// Composable
@Composable
fun InstrumentsScreen(viewModel: InstrumentsViewModel = viewModel()) {
    val instruments = viewModel.instruments
    LaunchedEffect(Unit) { viewModel.loadInstruments() }
    LazyColumn {
        items(instruments, key = { it.id }) { instrument ->
            Text(text = instrument.name, modifier = Modifier.padding(8.dp))
        }
    }
}

```

15. Ejecutar la aplicacion

Ejecute la app en un emulador o dispositivo fisico usando **Run app** en Android Studio. Si todo fue configurado correctamente, deben mostrarse los tres registros de ejemplo:

- **violin**
- **viola**
- **cello**

16. Flujo resumido

Tabla 4 Flujo paralelo de configuracion en Supabase y Android

#	En Supabase	En Android
1	Crear proyecto	Crear proyecto Kotlin
2	Crear tabla <code>instruments</code>	Agregar dependencias
3	Insertar datos de ejemplo	Agregar permiso de Internet
4	Activar RLS	Inicializar cliente Supabase
5	Crear politica de lectura	Crear modelo <code>Instrument</code>
6	Exponer tabla en Data API	Consultar desde ViewModel
7	Obtener URL y publishable key	Mostrar datos con Compose

17. Problemas mas probables y soluciones

La tabla existe pero no devuelve datos

- Verifique que ejecuto `grant select on public.instruments to anon;`
- Verifique que activo RLS con `alter table instruments enable row level security;`
- Verifique que creo la politica de lectura.

La app no conecta con Supabase

- Revise `supabaseUrl` (debe incluir `https://`).

- Revise `supabaseKey` (debe ser la publishable key, no la service_role).
- Revise que el permiso `INTERNET` esta en `AndroidManifest.xml`.

La consulta compila pero falla al decodificar

- Confirme que el modelo Kotlin coincide exactamente con los campos de la tabla: `id` y `name`.
- Revise la anotacion `@Serializable` en la data class.

La app abre pero la lista aparece vacia

- Revise si inserto los datos de prueba con el script SQL.
- Revise si la tabla fue expuesta en **Integrations > Data API**.
- Verifique que `loadInstruments()` esta siendo llamado desde el composable.

PARTE II

Diagrama CRUD con ViewModel y Supabase

Arquitectura de flujo unidireccional: UI — ViewModel — Estado — Supabase

1. Diagrama de arquitectura CRUD

Este diagrama ilustra el patrón de flujo de datos unidireccional (UDF) que conecta la interfaz de usuario con Supabase a través del ViewModel. Cada capa tiene una responsabilidad única y no puede ejecutar la de las demás.

Conexión de Android + ViewModel + Supabase en un CRUD

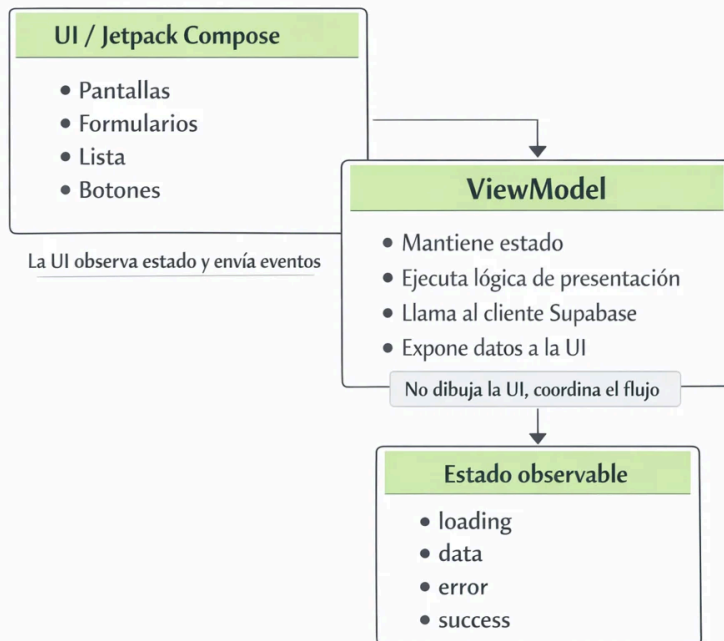


Figura 1. Diagrama de arquitectura: UI / Jetpack Compose — ViewModel — Estado Observable — Cliente Supabase — PostgreSQL

MODELO MENTAL DEL DIAGRAMA

La UI pide → el ViewModel decide → Supabase ejecuta → la UI refleja el resultado.

La interfaz no habla directamente con la base de datos. La UI emite eventos, el ViewModel organiza la logica, el cliente de Supabase ejecuta la operacion y el resultado regresa como un nuevo estado que la UI vuelve a representar.

2. Lectura de cada capa del diagrama

2.1 UI / Jetpack Compose

La UI es la capa visible. Su trabajo no es decidir como consultar la base de datos ni interpretar reglas de negocio. Su funcion es: capturar acciones del usuario (toques, formularios, botones), mostrar listas y pantallas, y observar cambios de estado para recomponerse automaticamente.

2.2 ViewModel

El ViewModel es la pieza de coordinacion. No dibuja la pantalla, pero organiza el comportamiento. Recibe eventos de la UI, decide que operacion debe ejecutarse y actualiza el estado observable. Sus responsabilidades en el diagrama son:

- **Mantiene estado:** la lista de datos sobrevive rotaciones de pantalla y cambios de configuracion.
- **Ejecuta logica de presentacion:** decide que operacion CRUD lanzar en respuesta a cada evento.
- **Llama al cliente Supabase:** inicia las peticiones de red mediante coroutines.
- **Expone datos a la UI:** publica el nuevo estado para que la UI lo observe.

PRINCIPIO CLAVE

El ViewModel es el director del proceso de datos. No dibuja la UI ni coordina vistas directamente.

2.3 Estado observable

Representa la estructura de datos que el ViewModel expone a la UI, con cuatro dimensiones fundamentales:

loading

Operacion en curso. La UI puede mostrar un indicador de carga.

data

Datos validos recibidos desde Supabase. La UI construye su lista a partir de este campo.

error

La operacion fallo. La UI puede mostrar un mensaje de error al usuario.

success

Una operacion de escritura (insert, update, delete) termino correctamente.

2.4 Cliente Supabase + PostgREST

Este bloque encapsula la comunicacion con Supabase. El ViewModel solicita una operacion y el cliente la transforma en una llamada HTTP hacia la API. Las operaciones disponibles son: `createSupabaseClient()`, `from("tabla")`, `select()`, `insert()`, `update()` y `delete()`.

2.5 Supabase API y PostgreSQL

La API funciona como capa de acceso controlado. Recibe la solicitud del cliente Android, verifica la ruta de la operacion y la ejecuta sobre la base de datos conforme a las reglas configuradas, incluyendo las politicas RLS. La base PostgreSQL almacena los datos reales y aplica las restricciones de seguridad.

3. Flujo de conexion paso a paso

Tabla 5 Los ocho pasos del flujo CRUD en el diagrama

Paso	Flujo	Explicacion
1	UI → ViewModel	El usuario toca un boton o envia un formulario. La UI convierte esa accion en un evento.
2	ViewModel → Cliente	El ViewModel interpreta la intencion y dispara la operacion CRUD correspondiente.
3	Cliente → Supabase API	El cliente envia una solicitud HTTP a la API de PostgREST de Supabase.
4	API → PostgreSQL	La API ejecuta la operacion sobre la base y valida permisos mediante RLS.
5	PostgreSQL → API	La base devuelve el resultado de la consulta o modificacion.
6	API → Cliente	La respuesta vuelve en formato JSON hacia el cliente Android.
7	Cliente → ViewModel	El cliente decodifica el JSON y lo entrega como objetos tipados de Kotlin.
8	ViewModel → UI	El ViewModel actualiza el estado observable y la UI se recompone automaticamente.

4. Las cuatro operaciones CRUD desde el ViewModel

4.1 CREATE — Crear registro

Secuencia: la UI envia un formulario → el ViewModel valida la informacion → ejecuta `insert()` → Supabase guarda el registro → la UI recibe un estado de exito.

4.2 READ — Leer registros

Secuencia: la UI abre una pantalla → el ViewModel llama `select()` → Supabase responde con los registros → la UI muestra la lista. Esta es la operacion mas visible y la base de los quickstarts de Supabase.

4.3 UPDATE — Actualizar registro

Secuencia: la UI edita un campo → el ViewModel envia `update()` → Supabase actualiza el contenido → la UI refresca su estado. El cambio se refleja por emision de un nuevo estado, no por manipulacion manual de la vista.

4.4 DELETE — Eliminar registro

Secuencia: la UI confirma un borrado → el ViewModel ejecuta `delete()` → Supabase elimina el registro → la UI remueve el elemento de la lista. La eliminacion tambien es un flujo de estado, no una accion visual aislada.

5. Seguridad y acceso

- **Publishable key en el cliente:** la app movil usa una clave publica adecuada para consumo desde el lado cliente.
- **RLS controla permisos:** Row Level Security define que filas pueden verse o modificarse segun el rol del usuario.
- **Sin claves de servidor en la app:** las claves con privilegios elevados no deben exponerse en una aplicacion movil.

Tabla 6 Resumen de responsabilidades de cada capa

Capa	Pregunta que responde	Responsabilidad
UI	Que ve y hace el usuario?	Captura eventos y representa estado.
ViewModel	Que debe pasar ahora?	Coordina logica de presentacion y solicitudes CRUD.
Cliente Supabase	Como se formula la operacion?	Convierte la intencion en una llamada de red valida.
Supabase API	Como se accede de forma controlada?	Intermedia y aplica reglas de acceso (RLS).
PostgreSQL	Donde viven los datos?	Almacena, consulta y modifica registros.

PARTE III

Diagrama Tecnico: Flujo desde Activity hasta Supabase

Analisis tecnico completo del patron MVVM en Android con Supabase como backend

1. Diagrama tecnico completo

Este diagrama muestra el flujo tecnico completo de una aplicacion Android desde su punto de entrada (`MainActivity`) hasta los servicios del backend de Supabase, pasando por el composible, el ViewModel y la capa de comunicacion HTTP/JSON.

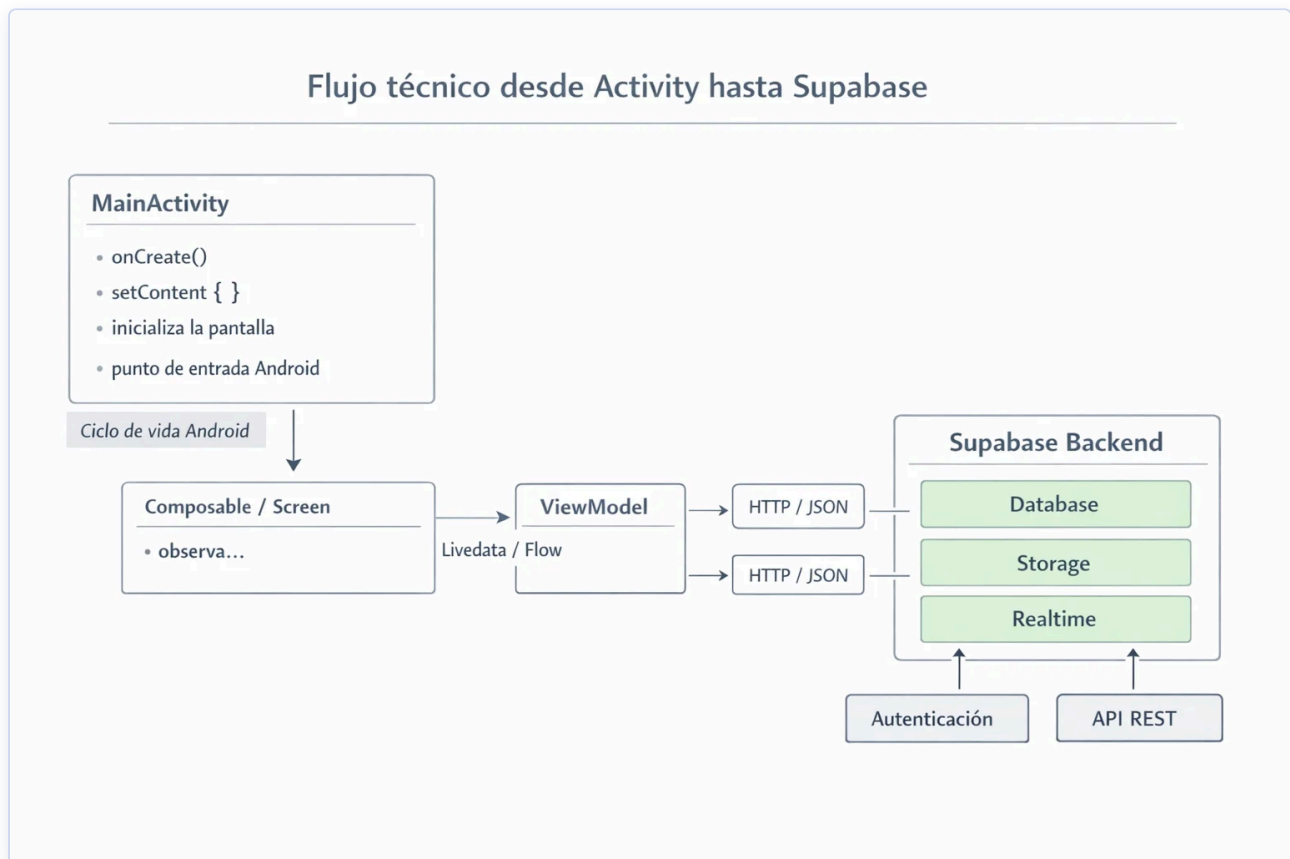


Figura 2. Diagrama tecnico: flujo desde `MainActivity` → Composable → ViewModel → HTTP/JSON → Supabase Backend (Database, Storage, Realtime, Auth, API REST)

2. Analisis de capas y componentes

2.1 MainActivity — Punto de entrada Android

La `MainActivity` es el componente fundamental de Android que actua como contenedor de la interfaz de usuario y es el punto donde el sistema operativo entrega el control a la aplicacion.

- `onCreate()` : metodo del ciclo de vida invocado al crear la actividad. Aqui se realiza toda la configuracion inicial.
- `setContent { }` : funcion de Jetpack Compose que declara el arbol de UI de forma programatica.

CICLO DE VIDA ANDROID

La flecha "Ciclo de vida Android" que apunta desde `MainActivity` hacia `Composable / Screen` representa que el sistema operativo controla cuando la actividad se crea, se detiene o se destruye. El composable vive dentro de ese ciclo de vida.

2.2 Composable / Screen — Capa de presentacion

El composable es la funcion de Jetpack Compose que construye la pantalla visible. Su unica responsabilidad es la representacion visual del estado. No contiene logica de negocio ni llamadas de red. Se suscribe a flujos de datos del ViewModel mediante `LiveData` o `StateFlow`.

2.3 ViewModel — Capa logica y puente de datos

Caracteristicas tecnicas del ViewModel en este contexto:

- **Sobrevive a cambios de configuracion:** cuando el usuario rota el dispositivo, Android destruye y recrea la Activity y el composable, pero el ViewModel permanece activo y los datos no se pierden.
- **Sin referencia a UI:** el ViewModel no tiene acceso a ningun widget de Android, lo que lo hace completamente testeable de forma independiente.
- **Expone streams de datos:** publica su estado mediante `LiveData` o `StateFlow` que el composable observa.
- **Ejecuta peticiones de red:** llama al cliente de Supabase mediante coroutines en el dispatcher de IO.

2.4 Comunicacion HTTP/JSON

La etiqueta **HTTP/JSON** entre el ViewModel y el backend especifica el protocolo y el formato:

- **HTTP:** todas las peticiones a Supabase son llamadas HTTPS estandar gestionadas transparentemente por `ktor-client-android`.
- **JSON:** formato estandar de datos. La libreria `postgrest-kt` serializa objetos Kotlin a JSON al enviar y los deserializa al recibir, usando las anotaciones `@Serializable`.

3. Flujo tecnico completo de extremo a extremo

El siguiente recorrido describe los nueve pasos tecnicos desde que el usuario abre la app hasta que los datos se muestran en pantalla:

- 1 **Sistema operativo inicia MainActivity.** Android llama a `onCreate()`, que invoca `setContent()` para construir el arbol de composables.
- 2 **Composable se monta y obtiene referencia al ViewModel.** Jetpack Compose crea o recupera una instancia del ViewModel asociada al ciclo de vida de la pantalla.
- 3 **Composable se suscribe al estado del ViewModel.** A traves de `StateFlow` o `LiveData`, el composable queda en escucha de cualquier cambio de estado.
- 4 **ViewModel dispara la carga de datos.** Mediante `viewModelScope.launch()`, inicia una coroutine que ejecuta la peticion en el dispatcher de IO sin bloquear el hilo principal.
- 5 **Cliente Supabase construye la peticion HTTP.** La funcion `supabase.from("tabla").select()` se traduce en una peticion GET al endpoint de PostgREST del proyecto.
- 6 **Supabase API recibe y procesa la peticion.** El servidor verifica la autenticacion mediante la publishable key, aplica las politicas RLS y ejecuta la consulta en PostgreSQL.
- 7 **Supabase responde con JSON.** Los datos de la tabla se serializan en formato JSON y viajan de vuelta al cliente Android via HTTPS.
- 8 **Cliente deserializa la respuesta.** `decodeList<Instrument>()` convierte el JSON en una lista tipada de objetos Kotlin. El ViewModel actualiza su estado observable.
- 9 **Composable se recompone automaticamente.** Al detectar el cambio en el estado observado, Jetpack Compose redibuja los elementos afectados y muestra los datos al usuario.

4. Servicios del backend Supabase

El diagrama muestra el backend de Supabase como un conjunto integrado de servicios. Cada uno tiene un rol especifico en la arquitectura:

4.1 Database (PostgreSQL)

El nucleo del backend: una base de datos PostgreSQL completamente funcional donde se almacenan los datos estructurados. Incluye soporte completo para Row Level Security, transacciones, funciones almacenadas y triggers.

4.2 API REST (PostgREST)

Supabase genera automaticamente una API REST completa para cada tabla expuesta en la Data API. El cliente `postgrest-kt` en Android se comunica exclusivamente con esta API. Cada operacion (`select` , `insert` , `update` , `delete`) se transforma en la llamada HTTP correspondiente.

4.3 Autenticacion

El sistema de autenticacion de Supabase gestiona usuarios, sesiones y tokens. Para aplicaciones con usuarios autenticados, Supabase provee JWT que se incluyen automaticamente en las cabeceras de cada peticion, permitiendo que RLS identifique al usuario y aplique las politicas correspondientes.

4.4 Storage

Servicio para almacenamiento de archivos binarios (imagenes, videos, documentos) que no son apropiados para una base de datos relacional. El cliente Kotlin puede interactuar con Storage usando el modulo `storage-kt` .

4.5 Realtime

Permite recibir cambios de la base de datos en tiempo real sin necesidad de polling. El cliente Android puede suscribirse a los cambios (INSERT, UPDATE, DELETE) mediante WebSockets y actualizar la UI instantaneamente. Ideal para aplicaciones de chat, dashboards en vivo o funcionalidades colaborativas.

Tabla 7 Servicios del backend Supabase y su rol en la arquitectura Android

Servicio	Protocolo	Uso principal en Android
Database (PostgreSQL)	Interno	Almacenamiento de datos estructurados; objetivo de las operaciones CRUD.
API REST (PostgREST)	HTTPS + JSON	Endpoint que <code>postgrest-kt</code> consume para todas las operaciones de datos.
Autenticacion	HTTPS + JWT	Gestion de usuarios, sesiones y permisos via tokens JWT.
Storage	HTTPS	Subida y descarga de archivos binarios.
Realtime	WebSocket	Suscripcion a cambios de la base de datos en tiempo real.

5. Patron MVVM en detalle

El diagrama es una implementacion concreta del patron **Model-View-ViewModel (MVVM)**:

Tabla 8 Mapeo del patron MVVM a los componentes del diagrama tecnico

Capa MVVM	Componente en el diagrama	Responsabilidad tecnica
View	<code>MainActivity</code> + <code>Composable</code>	Presenta la interfaz. Observa el estado del ViewModel. No contiene logica de datos.
ViewModel	<code>ViewModel</code>	Mantiene el estado. Ejecuta operaciones. Expone streams observables. Sobrevive a rotaciones.
Model	<code>Supabase Backend</code> + <code>postgrest-kt</code>	Fuente de verdad de los datos. El cliente abstrae la comunicacion con el backend.

Por que MVVM con Supabase?

- **ViewModel** resuelve el problema del ciclo de vida: los datos no se pierden cuando el usuario rota el dispositivo o cambia de pantalla.
- **Supabase** resuelve el problema del backend: toda la infraestructura de base de datos, autenticacion y almacenamiento sin implementar un servidor propio.
- **Jetpack Compose** resuelve el problema de la sincronizacion UI-datos: el enfoque declarativo y reactivo hace que la UI se actualice automaticamente cuando el estado del ViewModel cambia.

PRINCIPIO DE SEPARACION DE PREOCUPACIONES

El patron completo garantiza que ninguna capa tenga que conocer los detalles internos de las demas. La UI no sabe de HTTP. El ViewModel no sabe de widgets. Supabase no sabe de Android. Cada capa puede cambiar, testearse o reemplazarse de forma independiente.

Conclusion General

Este documento ha presentado de forma integral la arquitectura completa para integrar Supabase a una aplicacion Android en Kotlin con ViewModel. Las tres partes son complementarias y cada una agrega una dimension diferente de comprension:

- **El manual paso a paso** (Parte I) provee las instrucciones concretas y el codigo minimo funcional para implementar la integracion desde cero, siguiendo la documentacion oficial de Supabase.
- **El diagrama CRUD con ViewModel** (Parte II) explica el flujo conceptual: como la UI, el ViewModel y Supabase colaboran en un patron unidireccional donde cada capa tiene una responsabilidad unica.
- **El diagrama tecnico desde Activity** (Parte III) profundiza en la implementacion tecnica: desde el ciclo de vida de Android hasta los servicios especificos del backend de Supabase.

Principios fundamentales de la arquitectura

- La **UI no gestiona la base de datos**: solo observa el estado y emite eventos.
- El **ViewModel no dibuja pantallas**: coordina la logica de presentacion y las llamadas de datos.
- **Supabase no decide la experiencia visual**: provee datos y servicios de infraestructura.
- La **Publishable key protege el acceso**: nunca debe usarse la service_role key en codigo cliente.
- **RLS es la linea de defensa en la base de datos**: controla que filas puede ver o modificar cada usuario.

Cuando cada capa conserva su papel, el CRUD deja de ser una secuencia confusa de llamadas y se convierte en un circuito entendible, mantenible y escalable que puede crecer con la aplicacion.

PROXIMOS PASOS RECOMENDADOS

Una vez dominada esta base: implementar autenticacion de usuarios con Supabase Auth, extender el ViewModel con `StateFlow` para un estado mas robusto, agregar las operaciones de escritura (INSERT, UPDATE, DELETE) al ViewModel, y explorar las capacidades de Realtime para actualizaciones en vivo.

Documento generado a partir de la documentacion oficial de Supabase para Android Kotlin (supabase.com/docs/guides/getting-started/quickstarts/kotlin), complementado con diagramas de arquitectura y analisis tecnico detallado de cada capa del patron MVVM aplicado a la integracion con Supabase.